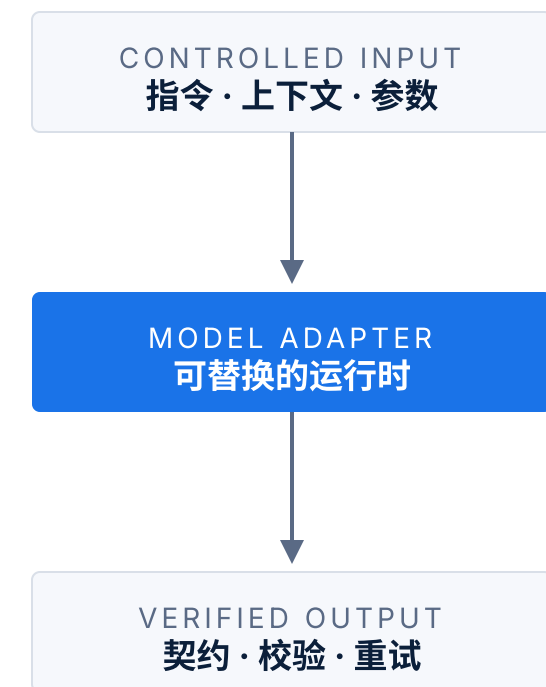


从模型 API 到可控的 Agent Harness

不是「会调模型」，而是让模型调用成为**可替换、可校验、可恢复**的工程能力。



🕒 HARNESS — 三件事都受控

SCOPE Model API · Parameters · Failure Boundaries | FOCUS Engineering Discipline | PREREQ Chapter 1.1

LLM 统一模型调用服务的第一块能力

在把 Agent Loop 跑起来之前,我们必须先解决一个更基础的问题:如何稳定调用模型。

TRADITIONAL · HTTP API

请求确定,
响应可预测

```
POST    /api/login
Content-Type: application/json
{ "user": "...", "pwd": "..."}
→ { "code": 0, "token": "..."}

```

LLM · PROBABILISTIC API

请求是上下文,
响应是概率生成

```
def
create(messages, temperature, ...)
⇒ 概率最大的下一段文本
① 同样的输入,可以得到不同的输出

```

本节解决 如何稳定调用模型 封装输入 · 控制参数 · 输出契约 · 失败分类 · Adapter 抽象	后续章节解决 流式交互 · Prompt · Tool Use · Loop 建立在这一层之上,本章先把地基打稳
---	---

今天只做六件事

每件事都是一个可验证、可演进的工程能力,而不是一个 demo 步骤。

01

理解 Chat / Responses API 的结构

识别请求字段、消息角色与响应体的来源,而不是靠记忆调用姿势。

02

理解三大采样参数的边界

`temperature`、`top_p`、`max_tokens` 各控制什么,不被「调大温度」误解。

03

管理 Token 与 Context Window

为模型喂「足够且必要」的上下文,而不是把全部历史塞进 prompt。

04

建立 Structured Output 契约

JSON Schema / Pydantic / 原生结构化输出:从「看起来像 JSON」到业务可校验。

05

分类处理模型调用失败

超时 · 限流 · 过载 · 拒答 · 截断 · 格式错,不同失败对应不同处置。

06

切换至少两个模型 Adapter

不改 Loop 就能换模型,关键能力可路由、可回放、可回归。

AI 写代码,工程师还负责什么?

不是分工对立,而是职责重新划分:生成候选结果交给模型,让候选结果可用仍由工程师定义。

MODEL

在给定上下文中生成候选结果

写一段文本、生成一段代码、提出一种修改路径

ENGINEER

让候选结果在系统中可用

目标可验收 · 约束可执行 · 失败可定位 · 变更可回滚

工程师必须指定	不能交给模型自行决定的原因
任务目标	「写代码」不等于可验收的修改范围 —— 模型不知道这单 PR 要不要附带测试。
上下文	模型不知道当前仓库、约束、依赖和已发生的历史决策。
输出契约	自然语言不能安全地驱动程序分支或工具调用,必须有结构化约束。
权限与预算	模型不能决定自己能花多少钱、能改哪些文件 —— 这些必须由系统授权。
失败边界	模型不知道何时重试、何时降级或必须停止 —— 这是工程师的设计责任。

结论 · 编码可以交给 AI;目标、上下文和边界必须由工程师定义。

为什么输入输出是 Agent 工程的起点

INPUT · 受控请求

输入不是一句 Prompt

而是一次受控请求,包含三件事:指令、上下文、参数。

指令	系统规则与任务目标
上下文	代码、文件摘要、历史消息、工具结果
参数	随机性、输出预算、模型能力

OUTPUT · 下一步依据

输出不是一段文本

而是下一步系统行动的依据。

- ? 给用户展示?
- D 写入数据库?
- T 调用工具?
- F 修改文件?
- ↺ 进入下一轮 Loop?

输入定义可做什么 · 输出定义系统如何行动

两件事都在 Harness 内部显式声明,而不是默认靠字符串解析。

Chat Completions 与 Responses

从聊天接口到 Agent 运行时 —— 不是新旧替代,而是两种不同的心智模型。

CHAT COMPLETIONS

围绕 messages 的请求—响应

- 1

心智模型简单

输入是一个 messages 数组,输出是一段文本。
- 2

生态与兼容实现广泛

OpenAI Compatible 协议、第三方代理、本地推理服务都默认支持。
- 3

适合多供应商统一接入

共同基线:Chat API 是大多数模型能保证的稳定形态。

```
response = client.chat.completions.create(
    model="deepseek-v4-flash",
    messages=[{ "role": "system", ... }, ...]
)
```

RESPONSES

面向输出项的统一接口

- 1

多模态输入

文本、图片、文件、检索结果可同时传入,统一 item 化。
- 2

工具调用与事件

function_call / reasoning / refusal 都是显式输出项。
- 3

新 Agent / Codex 首选

适合需要丰富语义的新建 Agent 应用。

```
response = client.responses.create(
    model="deepseek-v4-flash", input=[...])
for item in response.output:
    print(item.type, item)
```

ENGINEERING SELECTION

兼容层以 Chat 为基线 · 需要丰富 Agent 语义时研究 Responses

Chat Completions — 请求与响应

OpenAI Compatible SDK 调用 DeepSeek V4 Flash · 关注请求形状与响应字段来源。

```
main.py · OpenAI Compatible · DeepSeek V4 Flash

from openai import OpenAI

client = OpenAI(
    api_key=os.environ["DEEPSEEK_API_KEY"],
    base_url="https://api.deepseek.com",
)

response = client.chat.completions.create(
    model="deepseek-v4-flash",
    messages=[
        # system: 不可随意覆盖的全局契约
        {"role": "system", "content": "你是代码审查助手。"},
        {"role": "user", "content": "检查这段 Python 是否有空指针风险。"},
    ],
    max_tokens=600,
)

text = response.choices[0].message.content
```

REQUEST · 请求形状

请求是 messages 数组

保持 role 与 content 来源可识别, 这是后续裁剪/审计/重放的前提。

RESPONSE · 取值路径

choices[0].message.content

先读 choices,再读 message,最后才是 content。
这是响应的「唯一可信入口」。

关键不是 SDK,而是我们接下来要封装的输入、输出和错误边界。

消息不是聊天记录,而是运行时上下文

每一种 role 都承担一种工程职责。混进 user 字符串是埋雷的第一步。

ROLE	工程含义
system	不可随意覆盖的 全局行为契约 。模型的角色、能力边界和保守度,都从这里开始。
user	用户目标、输入数据与约束。 这是唯一面向模型的「用户输入」位置。
assistant	模型已经作出的决定,支持 多轮连续性 。下一轮前的状态、修订、撤回都靠它保留。
tool	外部世界的观察结果 ,必须可追溯。工具是模型的「传感器」,它决定下一步能看见什么。

DON'T · 反例

把系统规则、用户输入、工具结果、历史结论拼成一大段字符串

无法裁剪 · 无法审计 · 无法重放 · 无法证明「模型知道什么」

DO · 正解

保留来源、role 与顺序,让 Harness 能够裁剪、审计与重放

messages 是一个**运行时状态数组**,不是 UI 上的对话流。

为什么 Agent 需要「输出项」而不仅是文本

Agent 关注的不是「模型说了什么」,而是「系统下一步要处理什么」。

```
responses_api.py

response = client.responses.create(
    model="deepseek-v4-flash",
    input=[
        {"role": "user", "content": "为 login 函数生成测试边界。"},
    ],
)

for item in response.output:
    print(item.type, item)
```

OUTPUT · next-action items

system.next_action()

message

可展示的回答

function_call

待执行的工具请求

reasoning

推理项

refusal

拒答

KEY TAKEAWAY

模型输出是运行时事件,不是只能 `print()` 的字符串

WHY THIS MATTERS

Tool Use 与 Loop 后续消费这些输出项。今天先建立认识 —— 把模型当成事件源,而不是 chat 对象。

一个简单的判断标准:Loop 是否需要根据输出决定下一步 —— 是 → 输出项;否 → 纯文本。

参数不是「质量旋钮」

COMMON MISCONCEPTION

回答不满意,就把 `temperature` 调高

参数	真正控制的对象	不解决的问题
<code>temperature</code>	采样随机性 · 同输入可出现不同输出	事实正确性 · 上下文缺失 · 工具失败
<code>top_p</code>	候选 token 的概率质量范围	任务理解 · 输出契约
<code>max_tokens</code>	单次输出预算上限 · 决定何时停止生成	上下文容量 · 思考质量 · 整体任务理解

PRIORITY ORDER

质量问题先检查 → 目标 · 上下文 · 模型能力 · 工具证据 · 输出约束

参数永远排在前面五项之后。

如果调高温度救不了你的输出,问题几乎一定出在 prompt / 上下文 / schema / 工具证据 —— 而不在参数上。

temperature — 控制随机性,不是控制聪明程度

```
temperature.py

client.chat.completions.create(
    model="deepseek-v4-flash", messages=messages,
    temperature=0.2,
)
```

OUR DEFAULT CHOICE

T = 0.2

稳定、可复现、回归可批。

LOW · 0.0 – 0.3

更稳定、更易复现

适合:分类 · 抽取 · 代码修改计划 · 任何被程序消费的输出

降低回归波动 · 输出更收敛

HIGH · 0.7 – 1.0

更多表达与路径探索

适合:头脑风暴 · 文案备选 · 多种实现思路并行探索

代价:回归测试波动 · 不能直接被消费

ENGINEERING RULE

需要被程序消费、需要回归评测的输出 → 低随机性 + 明确契约

同一参数不一定被所有模型、所有模式支持或生效。调用前要查模型能力,而不是将某厂商经验复制到所有模型。

一些推理类 API 在 thinking 模式下不允许 temperature 修改 —— 当 T=0.2 没用时,先确认模型文档,再考虑调整其它维度。

top_p — 与 temperature 二选一作为主旋钮

```
awkward_knobs.py

# 默认策略:只控制一个采样维度
generation = {"temperature": 0.2, "max_tokens": 800}

# 只有在模型文档与实验均证明必要时改为:
# generation = {"top_p": 0.9, "max_tokens": 800}

# 不要同时大力度调节 — 难以解释、难以回归
```

SAMPLING CONTROL

一只旋钮

选择 temperature 还是 top_p:
→ 默认 temperature :可解释、好观测
→ 需要候选多样性时再切 top_p

WHY TWO KNOBS HURTS REPRO

同时大幅调节 temperature 与 top_p → 无法解释 · 无法回归

① 解释困难 ② 难做基线对照 ③ 输出分布不可复现 ④ 评审时无法定位是哪个变量在起作用

ENGINEERING RULE

一次实验只改变一个变量 · 参数版本与测试集绑定

像 git commit 一样管理参数:

gen-config-v3.json + regression-set-2026-08.json

max_tokens — 输出预算,更是停止条件

控制生成成本、延迟与被截断概率 —— 不是「我希望模型多写点」的意思。

```
max_tokens.py

result = client.chat.completions.create(
    model="deepseek-v4-flash", messages=messages,
    max_tokens=400,
)

if result.choices[0].finish_reason == "length":
    raise OutputTruncated("输出被预算截断,不能当作完整结果使用")
```

```
finish_reason == "length"
```

视为失败,继续重试

不要尝试「尽量解析」被截断的 JSON,残缺的结构化输出带来的代价远大于再次请求。

控制

本次输出最长多少 token

决定模型何时停止,直接关系:
成本 · 延迟 · 截断概率

影响

成本、延迟与截断

max_tokens ↑ ⇒ 单次费用 ↑ · 触发长度截断的概率 ↑
过小 ⇒ 任务无法完成 · 过大会消耗预算

不等于

Context Window 容量

输入 token 与输出 token 共同占用上下文,
输入越长 → 可用输出空间越少

关键 JSON 若因长度截断 → 视为失败,而不是「尽量解析」

truncated JSON 看起来像 JSON,但语义已经残缺。把截断当作错误信号,扩大预算或拆分任务再次生成。

输入越短,可分配给输出的预算越大 → 上下文预算需要为输出预留 token。

Token 与 Context Window — 容量不是无限记忆

ONE CALL'S BUDGET

$$\text{system} + \text{用户输入} + \text{历史消息} + \text{工具结果} + \text{预留输出} \leq \text{context window}$$

TRAP · 1

输入越长,可用输出越少

预设 max_tokens 不会随输入增加而自动增大。
结果:模型最后一次思考需要的输出空间不足。

TRAP · 2

工具结果挤掉任务信息

把搜索引擎全文、grep 输出、长日志塞进上下文,
任务指令与历史决策反而被挤出窗外。

TRAP · 3

超窗处理 → 粗暴截断

直接报错,或应用层把中间消息砍掉,
模型实际看到的是「残缺的世界」,任务失真。

HARNESS RESPONSIBILITY

给模型喂「足够且必要」的上下文

裁剪冗余 · 保留关键 · 预留输出 · 监控用量

上下文预算的最小实现

教学版按字符估算 · 生产环境必须基于目标模型 tokenizer,且为输出预留 token。

context_budget.py

```
def build_messages(history, task, max_history_chars=12_000):
    recent = []
    used = 0
    for message in reversed(history):
        size = len(message["content"])
        if used + size > max_history_chars:
            break
        recent.append(message)
        used += size

    return [
        {"role": "system", "content": "你是谨慎的 Coding Agent。"},
        *reversed(recent),
        {"role": "user", "content": task},
    ]

# 算法:倒序扫描 → 累计字节不超过 max_history_chars → 倒序复原 → 拼装为 system + 历史 + task
```

WHAT IT DOES

行为:倒序扫描历史,直到预算用尽;
原则:保留最近发生的事,丢掉久远的;
边界:system 与 user 始终保留。

WHAT'S MISSING

- 用字符而非 token 估算,不精确
- 没有为 max_tokens 预留输出空间
- 工具结果大小未单独控制
- 没有摘要降级策略

PRODUCTION NOTE

生产环境应基于**目标模型 tokenizer**计数,为**输出预留 token**,并采用**摘要 + RAG**降级策略。
教学版只是近似 —— 不要直接 copy 到生产。

为什么自然语言输出不能直接驱动程序

SCENARIO · 模型决定下一步是否修改文件

模型最终输出 —— 一段自然语言

「我建议先修复空值处理,应该可以改一下。」

运行时无法可靠判断的问题

?

改哪个文件？

「修复空值处理」指向哪一个？

?

改什么？

如果某人误解了「空值」,就改了不该改的。

?

是否已完成？

是否真的把那个文件改了,还是只是说说？

?

风险等级？

「应该可以」无法换算成风险评分。

THE FIX

把「模型意见」变成「可验证的协议数据」

下一张 slide 给出 Schema 三层防线。

三层 Structured Output — 约束 · 校验 · 业务防线

```
schema.py

from pydantic import BaseModel, Field

class PatchPlan(BaseModel):
    action: str = Field(pattern="^(edit|ask_user|finish)$")
    file_path: str | None = None
    reason: str = Field(min_length=1, max_length=500)
    risk: str = Field(pattern="^(low|medium|high)$")
```

SEE THE PIPELINE · 看到契约

一个 PatchPlan 物件

- action ∈ {edit, ask_user, finish}
- file_path 可空
- reason 不为空 · 至多 500 字
- risk ∈ {low, medium, high}

LAYER 1 · JSON MODE	
保证「看起来像 JSON」	⚠ 只有 JSON 语法，没有语义。
不保证字段正确 · 字段类型不确定 · 字段值是否合理未知。	
LAYER 2 · JSON SCHEMA / NATIVE STRUCTURED OUTPUT	
把结构约束交给模型服务	→ Field(pattern=...) → OpenAI structured outputs
字段名 · 类型 · 枚举 · 必填 · 长度范围都由模型在生成时遵循,失败直接 retry。	
LAYER 3 · PYDANTIC / BUSINESS RULES	
业务防线,在本地拒绝不安全的数据	
路径白名单 · 业务权限 · 字段语义合理性 · 跨字段不变量 —— 这是模型的「语义护栏」。	

AI 为什么不能替你定义 Schema

AI 无法承担的责任

×

无法承担字段含义、权限范围与向后兼容责任

「这个字段向后兼容吗?」是工程责任,不是概率问题。

×

无法决定「缺失 file_path 时能否执行」

业务规则一旦随模型变化,系统安全边界即被破坏。

×

无法证明「/etc/passwd 不应成为允许修改的路径」

路径白名单是组织决策,模型不知道什么应该被禁止。

AI 可以作的事

根据描述**生成 Schema 草案**,把字段名整理为合理命名,但最终**定义、采纳、拒绝**都归属于工程师。

```
business_guard.py

plan = PatchPlan.model_validate_json(model_text)

if plan.action == "edit" and not plan.file_path:
    raise ValueError("edit 动作必须提供 file_path")

if plan.file_path and plan.file_path.startswith("../"):
    raise ValueError("禁止越过工作区")
```

WHO DEFINES

工程师定义不变量

模型只能在不变量内
生成候选值。

模型调用失败 · 先分类,再决定是否重试

「重试」只对暂时性故障有效,不是对所有失败都好的兜底。

阶段	典型问题	是否重试	正确处置
请求前	参数 / Schema 不合法	否	本地修复,记录配置错误,不要发起请求。
认证	Key 无效 · 权限不足	否	终止并提示配置问题。
传输	网络中断 · 连接超时	有边界	短退避 · 保留幂等键 · 控制次数。
服务端	429 限流 · 5xx 过载	有边界	指数退避 + jitter · 减少对过载服务的二次伤害。
模型输出	拒答 · 被截断 · 格式错	视原因	修复输入 · 调整预算 · 修正 Schema。不要盲目重试。

重试的是暂时性故障,不是所有失败。

有边界的重试、退避与降级

Max attempts · exp backoff · jitter · 三者缺失任何一个都会让重试变成事故制造机。

```
retry.py

import random, time

TRANSIENT = (TimeoutError, ConnectionError,
              RateLimitError, ServiceOverloaded)

def call_with_retry(call, attempts=3):
    for attempt in range(attempts):
        try:
            return call()
        except TRANSIENT:
            if attempt == attempts - 1:
                raise
            delay = min(8, 0.5 * (2 ** attempt)) + random.uniform(0, 0.2)
            time.sleep(delay)
```

FOUR PIECES

① 最大尝试次数

防止任务无限等待 · 防止成本失控

② 指数退避

$2^{**} attempt$ ✓ 多服务器友好

③ Jitter 抖动

避免同时重试形成 thundering herd

④ 降级

换模型 · 缩短上下文 · 转人工

DECODE THE FORMULA

$delay = min(8, 0.5 \times 2^{attempt}) + uniform(0, 0.2)$

attempt=0 → 0.5–0.7s · attempt=1 → 1.0–1.2s · attempt=2 → 2.0–2.2s · 第三轮仍未恢复即终止。

退避有上限:过长的退避对单任务没意义,会让用户等待无法接受的总时长。

拒答、截断、格式错 · 不是同一种「失败」

模型层面的失败也分根因,不能泛化为「在 retry 不到的兜底里 cover 一切」。

现象	根因假设	下一步
内容拒答	任务触发安全边界或 上下文不充分	终止或请求用户澄清。 不能自动绕过。
finish_reason = "length"	输出预算不足	扩大预算或拆分任务, 再次请求。
JSON 解析失败	未开约束 · 输出被污染 · 输出被截断	重新开启结构化输出 → 仍失败则终止。
字段校验失败	模型生成值违反业务契约	回传具体字段错误 给模型; 限制重试次数。

PRINCIPLE

把失败原因写入 Trace,后续才能区分「模型问题」还是「系统设计问题」

trace_id · model_id · latency · finish_reason · schema_diff · retry_count

Model Adapter — 让 Loop 依赖能力,而不是依赖供应商

```
adapter.py

from typing import Protocol

class ChatModel(Protocol):
    def generate(self, messages: list[dict], *, max_tokens: int):
        ... # returns str

class DeepSeekChatAdapter:
    def generate(self, messages, *, max_tokens):
        response = self.client.chat.completions.create(
            model=self.model, messages=messages,
            max_tokens=max_tokens,
        )
        return response.choices[0].message.content or ""
```

CONTRACT SHOULD CARRY

return type
文本 · 结构化结果 · 工具请求

usage
token 用量 · 费用估算

finish_reason
正常 · 截断 · 拒答

DO NOT EXPOSE

- ❌ 某 SDK 的内部类型
- ❌ 嵌套三层 provider-specific 字段
- ❌ 不一致的命名约定

FUTURE EXTENSION

把返回类型扩展为统一的 `ModelResult` · 而不是直接返回 `str`。

THREE-LAYER ADAPTER PATTERN

- Agent Loop calls `chat_model.generate(...)`
- ChatModel Protocol defines a stable shape
- DeepSeekChatAdapter / OpenAIResponsesAdapter implements the shape

好处:换模型不改 Loop · 可写 FakeModel 用于测试 · 可加 Langfuse / OpenLLMetry。
代价:多一层抽象的初期学习成本 → 用业务单元测试覆盖即可回收。

不改 Loop 切换两个模型

Loop 只接受一个 `ChatModel`,换模型只是构造新对象。

```
mini_agent.py

class MiniAgent:
    def __init__(self, model: ChatModel):
        self.model = model

    def run(self, task: str) -> str:
        messages = build_messages([], task)
        return self.model.generate(messages, max_tokens=600)

agent = MiniAgent(model=DeepSeekChatAdapter(...))
# 切换时:
# agent = MiniAgent(model=OpenAIResponsesAdapter(...))
```

WHAT THIS BUYS YOU

① 供应商切换不污染 Loop

agent.run 还是那样;只是输入改了。

② 路由与 fallback

按成本、延迟、能力做选择。

③ FakeModel 测试

确定性单元测试 · 不依赖真实模型。

④ 可观测性

统一记录 token · 费用 · 错误 · trace。

ROUTING DIMENSIONS

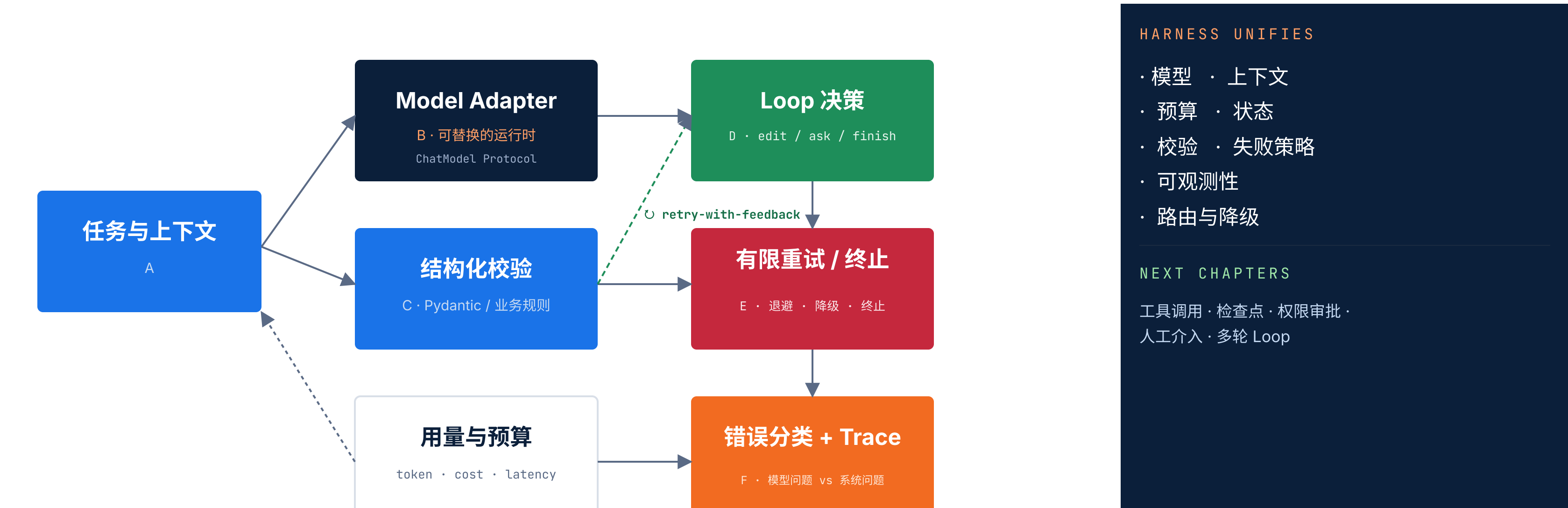
切换维度:成本 · 延迟 · 能力 · 区域可用性

按任务类型路由 · 按高峰期切到次级模型 · 跨区域容灾—这一切只发生在 Adapter 工厂,不在 Loop 内。

切换前应该先基于回归集评估,而不是凭直觉在线切换生产流量。

本节的 Harness 轮廓

Harness 不是一个 Prompt,也不是 SDK 包装 —— 它是一组协同的工程职责。



WHAT HARNESS IS NOT

✗ 一个 Prompt · ✗ 一个 SDK 包装

✓ 一组协同的工程职责 · 缺一不可。

Mini Coding Agent — 本节实战任务

TASK

输入一个代码问题 → 输出一个可校验的修改计划

INPUT

输入

问题描述

git issue 或 user question

仓库摘要

结构 · 依赖 · 现有测试

相关文件片段

不超过 token 预算

OUTPUT CONTRACT

输出契约

action ∈ 3 values

edit · ask_user · finish

file_path: string?

必须是工作区相对路径

reason: 1-500 字

为什么修改

risk ∈ 3 values

low · medium · high

EXCEPTIONS & SWITCH

异常与切换

429 / 超时 / 拒答

必须记录 + 有停止条件

Schema 校验失败

回传具体字段错误

模型切换

DeepSeek ↔ 别家 Adapter · Loop 不变

ACCEPTANCE

验收不是「模型回答得不错」

同类请求可回归 · 失败可定位 · 模型可替换 · 输出可校验

你现在具备的不是 「Prompt 调参」能力 而是模型运行时能力。

- 1 能读懂并封装主流模型 API**
Chat / Responses · 多供应商统一接入 · 不绑死某 SDK
- 2 能用实验而非感觉管理模型参数**
一次只改一个变量 · 参数版本与测试集绑定
- 3 能把模型输出变成受验证的数据契约**
JSON mode · Schema · Pydantic 业务护栏 三层防线
- 4 能设计错误分类、重试、退避、降级与终止边界**
「重试」是工具,不是兜底;每种失败有专名
- 5 能让模型供应商变化不影响核心 Loop**
Adapter 是契约,不是适配层